

GRAFI

INDICE SEZIONE

- ☐ DEFINIZIONE
- ☐ GRAFI SEMPLICI E NON DIRETTI
- ☐ VISITE (GRAPH TRAVERSAL)
- ☐ BREADTH FIRST SEARCH
- ☐ DEPTH FIRST SEARCH
- ☐ CONFRONTO BFS E DFS
- ☐ CAMMINI MINIMI SU GRAFI PESATI
- ☐ ALGORITMO DI DIJKSTRA
- ☐ GRAFI SEMPLICI DIRETTI
- ☐ VISITE SU GRAFI DIRETTI
- ☐ GRAFI DIRETTI ACICLICI

DEFINIZIONE

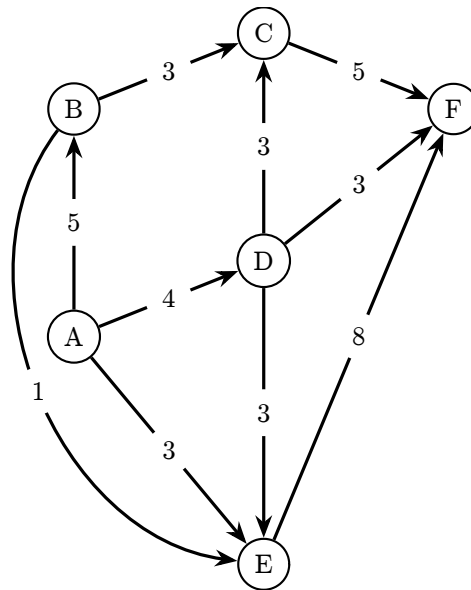
GRAFO

Grafo $G = (V, E)$:

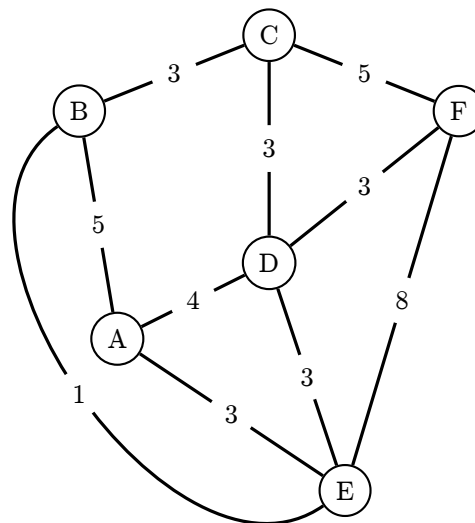
- V = insieme di **vertici** (o *nod*).
- E = collezione di **archi** (*coppie di vertici*).

Il grafo si dice **diretto** se ogni arco $(u, v) \in E$ è una *coppia ordinata* ($u \rightarrow v$), altrimenti si dice **non diretto** ($u - v$).

Esempio di grafo *diretto*:



Esempio di grafo *non diretto*:



🔥 OSSERVAZIONI

1. La definizione ammette la presenza di *archi multipli* tra due vertici (per questo E è una *collezione* e non un insieme) e di *self loop* (archi (u, u)).
2. Un *grafo semplice* è un grafo *senza archi multipli* (in questo caso E è un insieme) e *senza self-loop*.
3. Per alcune applicazioni, agli *archi* sono associati dei *pesi*.

Ci concentreremo principalmente su *grafi semplici*.

ESEMPI DI APPLICAZIONI

I grafi sono utilizzati di solito per rappresentare networked data, cioè dati in cui l'informazione è costituita da:

1. *Singoli elementi*.
2. **Relazioni** tra *(coppie)* di elementi.

NOTA

Esistono anche *generalizzazioni* dei grafi (ipergrafi) in cui le *relazioni* coinvolgono *più di 2 elementi*.

RETI SOCIALI

FACEBOOK:

- **Vertici**: *profili* degli utenti.
- **Archi**: *amicizie* fra utenti.
- **Non diretto**

INSTAGRAM:

- **Vertici**: *profili* degli utenti.
- **Archi**: *follow*
- **Diretto** (un utente può seguirne uno e non è detto il viceversa)

RETI STRADALI

- **Vertici**: *incroci*.
- **Archi**: *segmenti* di strade fra incroci.
- **Diretto** (possono esserci sensi unici).

ALTRI ESEMPI

- Internet
- Web
- Supercomputer
- Reti di comunicazione
- Protein-protein network.

GRAFI SEMPLICI E NON DIRETTI

🔗 TERMINOLOGIA

1. **Inglese**: per nodi/vertici si usa *nodes/vertices*. Per archi si usa *edges/arcs*.
Di solito si usa *edges* per grafi *non diretti*, *arcs* per grafi *diretti*.
2. Dato un arco $e = (u, v) \in E$ diciamo che e è *incidente* su u e v e che u e v sono *adiacenti*.
3. I *vicini* di un vertice v sono tutti i vertici u tali che $(u, v) \in E$.
4. Il *grado* di un vertice $v \in V$ ($\text{degree}(v)$) è il *numero di archi* incidenti su v .

CONCETTI FONDAMENTALI

📖 CAMMINO

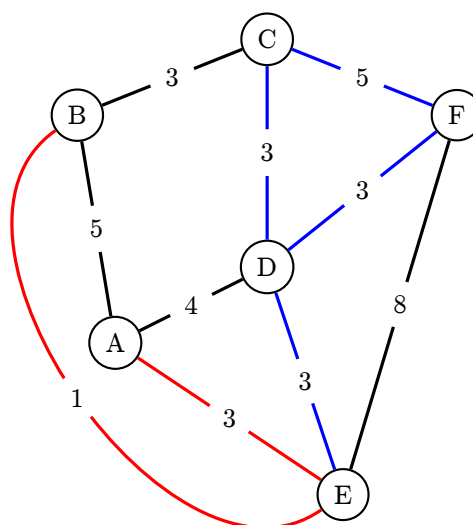
In inglese *path*:

$$u_1, u_2, \dots, u_k$$

con $(u_i, u_{i+1}) \in E$ per $1 \leq i \leq k$.

La *lunghezza del cammino* è il *numero di archi* (ovvero $k - 1$). Nel caso di *archi pesati*, la lunghezza è la *somma dei pesi degli archi*.

Il cammino si dice *semplice* se *non ha vertici ripetuti*.



In figura sono evidenziati due cammini:

- Cammino *semplice* **AEB** (in rosso).

- Cammino *non semplice* EDFCD (in blu).

CAMMINO MINIMO

Dati due vertici $x, y \in V$, il *cammino minimo* (**shortest path**) tra x e y , se ne esiste uno, è un cammino $x = u_1, u_2, \dots, u_k = y$ di *lunghezza minima*.

La sua lunghezza è detta **distanza** $d(x, y)$.

Se non esiste alcun cammino tra x e y , per convenzione si considera $d(x, y) = \infty$.

CICLO

E' un *cammino* $u_1, \dots, u_k$ con $u_1 = u_k$, cioè in cui i vertici *iniziale e finale coincidono*.

Un ciclo si dice **semplice** se gli unici vertici ripetuti sono gli estremi.

SOTTOGRAFO

Dato un grafo $G = (V, E)$, un suo *sottografo* è $G' = (V', E')$ con $V' \subseteq V$, $E' \subseteq E$ e tale che gli archi di E' incidono solo su V' .

SOTTOGRAFO DI COPERTURA

Dato un grafo $G = (V, E)$, un suo *sottografo di copertura* (**spanning subgraph**) è un sottografo $G' = (V', E')$ con $V' = V$ e $E' \subseteq E$.

Cioè un *sottografo* che *copre tutti i vertici*, ma con potenzialmente meno archi.

GRAFO CONNESSO

Un grafo $G = (V, E)$ si dice *connesso* (**connected**) se per ogni $u, v \in V$ esiste un *cammino* che inizia in u e termina in v .

GRAFO DISCONNESSO

Un grafo si dice *disconnesso* (**disconnected**) se *non è connesso*.

COMPONENTI CONNESSE

Le *componenti connesse* (**connected components**) di un grafo $G = (V, E)$ sono i *sottografi connessi massimali* di G , ovvero la famiglia di sottografi $G_i = (V_i, E_i)$ con $1 \leq i \leq k$ tali che:

- $G_i = (V_i, E_i)$ è *connesso* per $1 \leq i \leq k$.
- $V = V_1 \cup V_2 \cup \dots \cup V_k$ (partizione: $V_i \cap V_j = \emptyset \forall i \neq j$).
- $E = E_1 \cup E_2 \cup \dots \cup E_k$ (partizione: $E_i \cap E_j = \emptyset \forall i \neq j$)
- $\forall i \neq j$ non esistono archi in E tra V_i e V_j .

E si ha che se G è connesso $k = 1$.

ALBERO RADICATO (legame con i grafi)

Un *albero radicato* (rooted tree) è un grafo $G = (V, E)$ tale che:

- Esiste un vertice *radice* $r \in V$.
- Per ogni $u \in V$, con $u \neq r$, esiste un *unico padre* $p(u) \in V$, e si ha che $E = \{(u, p(u)) : u \in V, u \neq r\}$
- Per ogni $u \in V$ andando di *padre in padre* si *raggiunge* r .

ALBERO LIBERO (FREE TREE)

È un *grafo* $G = (E, V)$ *connesso e senza cicli*.

FORESTA

È un *grafo* $G = (E, V)$ *senza cicli*, ovvero un insieme di *alberi disgiunti*.

NOTA

Il concetto di albero radicato è **lo stesso** di quello già visto in > **ALBERI GENERALI**. I concetti di *albero radicato* e *albero libero* sono **equivalenti**: ogni albero *radicato* è un grafo connesso senza cicli, e ogni albero *libero* può essere visto come un albero radicato scegliendo un *vertice arbitrario* come *radice* e determinando di conseguenza le relazioni padre-figlio.

SPANNING TREE

Uno *spanning tree* di un grafo G è uno spanning subgraph *connesso e senza cicli* (free tree).

Chiaramente, uno spanning tree *esiste solo se G è connesso*.

SPANNING FOREST

E' uno spanning subgraph senza cicli.

Cioè è una famiglia di alberi che copre il grafo.

PRIMITIVE IMPORTANTI

- **Traversal**: esplorazione sistematica del grafo.
- **Connettività**: verifica se il grafo è connesso.
- **Identificazione componenti connesse**.
- **Ricerca di cammini minimi**.
- **Ricerca di minimum spanning tree**.
- **Stima della distanza media/massima**.

PROPRIETA'

PROPOSIZIONI

Sia $G = (V, E)$ un grafo *semplice e non diretto* con $|V| = n$ vertici ed $|E| = m$ archi. Valgono le seguenti proprietà:

1. $\sum_{v \in V} \text{degree}(v) = 2m$
2. $m \leq \binom{n}{2}$ e quindi $m \in O(n^2)$
3. Se G è un albero (rooted o free) allora $m = n - 1$.
4. Se G è connesso, allora $m \geq n - 1$.
5. Se G è senza cicli (cioè una foresta) $m \leq n - 1$.

DIM. 1

Osserviamo banalmente che ogni arco (u, v) è contato *esattamente due volte*: in $\text{degree}(u)$ e in $\text{degree}(v)$ $\square$.

✓ DIM. 2

Essendo G *semplice*, c'è *al più un arco per ogni coppia* (u, v) con $u \neq v$ e $u, v \in V$.
Perciò:

$$m \leq \binom{n}{2} = \frac{n(n-1)}{2} \in O(n^2)$$

✓ DIM. 3

Usando l'*equivalenza* tra *rooted* e *free tree* è sufficiente provare la proprietà in un rooted tree; e in un tale albero c'è *esattamente un arco* $(u, p(u))$ per ogni vertice $u \neq r \implies n - 1$ archi $\square$.

✓ DIM. 4

Consideriamo il ciclo seguente:

```
1: while esiste un ciclo C in G do
2:   rimuovi un arco da C
3: end while
```

Ad ogni iterazione il grafo *rimane connesso* e alla fine del while *non avrà cicli*.
Per cui, dopo l'eliminazione di alcuni archi saranno rimasti $n - 1$ archi.
Allora inizialmente il numero di archi doveva essere $\geq n - 1$ $\square$.

✓ DIM. 5

Sia $k \geq 1$ il numero di *componenti connesse* di G :

$$G_i = (V_i, E_i) \quad i = 1, \dots, k$$

$\forall i$ G_i è un grafo connesso e senza cicli.

Se $n_i = |V_i|$ e $m_i = |E_i|$ allora:

$$m = \sum_{i=1}^k m_i = \sum_{i=1}^k (n_i - 1) = -k + \sum_{i=1}^k n_i = n - k$$

Essendo $k \geq 1$, si ha $m \leq n - 1$ $\square$.

RAPPRESENTAZIONE DI GRAFI

Sia $G = (V, E)$ un grafo con n vertici ed m archi.

Spesso, per comodità, si rappresentano i *vertici* con gli *interi* da 1 a n .

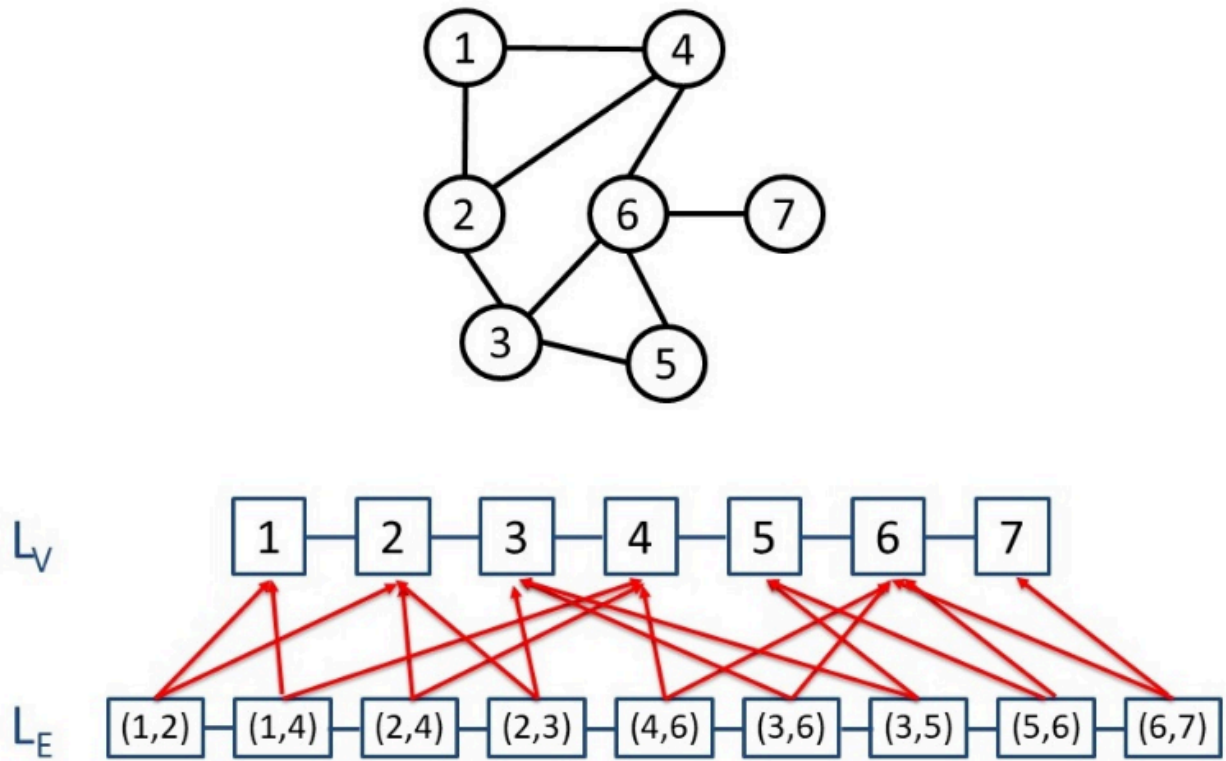
STRUTTURE DI BASE

La rappresentazione *più semplice* di G fa uso di:

- Una **lista di vertici** L_V : ogni nodo della lista contiene *tutte le informazioni* rilevanti per un *vertice* $v \in V$ *distinto*.
Se si rappresentano i vertici con numeri interi, L_V può essere un array.
- Una **lista di archi** L_E : ogni nodo della lista contiene *tutte le informazioni* rilevanti per un *arco* $e = (u, v) \in E$ *distinto*, includendo tra esse i *puntatori* a u e v .

NOTA

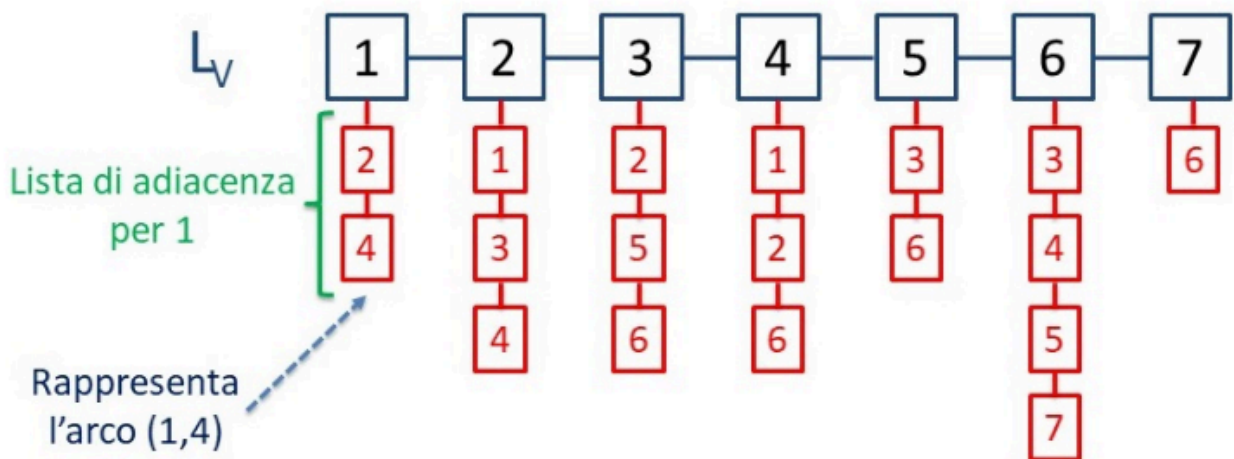
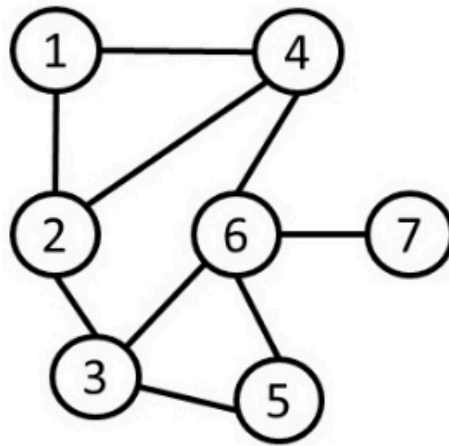
A seconda del problema da risolvere, ciascun vertice/arco può essere *arricchito da campi* strumentali per l'algoritmo scelto.



RAPPRESENTAZIONE ARCHI: LISTE DI ADIACENZA

Rappresentare gli archi tramite L_E rende *poco efficienti* gli algoritmi che devono *esplorare i vicini di un vertice* o che devono avere accesso diretto agli archi.

Una possibile soluzione è data dalle **liste di adiacenza** (*adjacency lists*): per *ogni vertice* $v \in V$ si ha una *lista* $\mathcal{I}(v)$ di *puntatori agli archi* (elementi di E) *incidenti su* v .



🔗 OSSERVAZIONE

Tale rappresentazione è quella più utilizzata per i seguenti motivi:

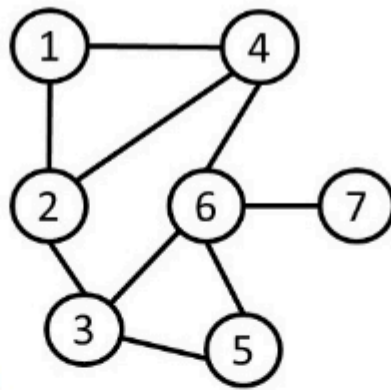
- Permette di rappresentare G in *spazio lineare* nella taglia del grafo, cioè $\Theta(n + m)$.
- Consente l'*accesso sequenziale* ai vicini di un vertice v , in *tempo lineare nel grado* di v .

RAPPRESENTAZIONE ARCHI: MATRICI DI ADIACENZA

Una soluzione alternativa è data dalle *matrici di adiacenza*: matrici $n \times n$ dove righe e colonne sono in corrispondenza 1 : 1 con i vertici e si ha:

$$A[i_1, i_2] = \begin{cases} \text{null} & \text{se } (i_1, i_2) \notin E \\ \text{puntatore a } e = (i_1, i_2) \in E & \text{se tale arco esiste} \end{cases}$$

Dato che il grafo è non diretto, rappresentano lo stesso arco



	1	2	3	4	5	6	7
1		(1,2)		(1,4)			
2	(2,1)		(2,3)	(2,4)			
3		(3,2)			(3,5)	(3,6)	
4	(4,1)	(4,2)				(4,6)	
5			(5,3)			(5,6)	
6			(6,3)	(6,4)	(6,5)		(6,7)
7						(7,6)	

🔥 OSSERVAZIONI

- La matrice di adiacenza consente l'*accesso* in *tempo costante* ad un arco e alle sue informazioni, ma richiede una rappresentazione dei *vertici tramite interi*.
- La matrice di adiacenza occupa *spazio quadratico* $\Theta(n^2)$.
Per questo motivo è una rappresentazione usata soprattutto nel caso di *grafi densi* (con numero di archi quadratico nel numero di vertici) o per grafi con *pochi vertici*.

MAPPE DI ADIACENZA

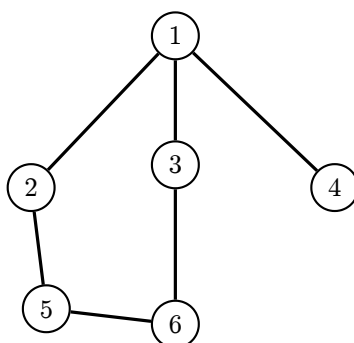
Una *via di mezzo* tra *liste* e *matrici* di adiacenza è costituita dalle **mappe di adiacenza**: per ogni vertice v gli archi incidenti su di esso sono memorizzati in una mappa. Usando una *tabella hash* per ogni mappa si può quindi avere *accesso* ad un qualsiasi arco in *tempo medio* $O(1)$, utilizzando globalmente uno spazio lineare nella taglia del grafo.

VISITE (GRAPH TRAVERSAL)

Si tratta di *procedure sistematiche* per esplorare G a partire da un vertice s , visitando *tutti* i vertici.

Esistono due approcci principali:

- **Breadth-First Search (BFS)** : dopo la visita di un vertice si visitano *tutti i vicini* prima di passare ai *vicini dei vicini*.
- **Depth-First Search (DFS)** : dopo la visita di un *vertice*, si visita un *vicino*, poi un *vicino del vicino* ecc.



Vediamo l'ordine di visita dei vertici sul grafo di esempio in figura:

- **BFS**: 1, 2, 3, 4, 5, 6.
- **DFS**: 1, 2, 5, 6, 3, 4.

BREADTH FIRST SEARCH

📖 NOTAZIONI E DEFINIZIONI

Sia $G = (V, E)$ un grafo semplice, non diretto e non pesato:

- Dato $s \in V$, $C_s \subseteq G$ denota la *componente connessa* di G *contenente* s , costituita da tutti i vertici raggiungibili con cammini da s , e dagli archi si essi incidenti.
- Ricordiamo che dati due vertici $x, y \in V$ nella stessa componente connessa, la loro *distanza* $d(x, y)$ è definita come la *minima lunghezza di un cammino* da x a y . Se i vertici sono in componenti connesse distinte, si assume $d(x, y) = +\infty$.

💡 IPOTESI IMPLEMENTATIVE

- Per ogni vertice $v \in V$ esiste un campo $v.ID$ che vale 1 se v è stato *visitato*, e vale 0 altrimenti.
- Per ogni arco $e \in E$ esiste un campo $e.label$ che memorizza una opportuna *etichetta* oppure vale null se e non ha ancora etichetta.
- Per ogni $v \in V$ $incidentEdges(v)$ restituisce un *iteratore agli archi incidenti* su v (quindi alla sua lista di adiacenza), che possono essere *enumerati* in tempo

$\Theta(1)$ ciascuno.

- Per ogni arco $e = (v, w) \in E$ $\text{opposite}(v, e)$ restituisce w in tempo $\Theta(1)$.

ALGORITMO

Algoritmo iterativo che, a partire da un vertice s :

1. *Visita tutti i vertici* di C_s .
2. *Etichetta* ciascun arco di C_s come DISCOVERY o CROSS EDGE.
3. *Partiziona* i vertici di C_s in *livelli* L_i in base alla loro *distanza* i da s .

Algoritmo BFS(G, s)

- **INPUT**: grafo $G = (V, E)$, $s \in V$, ogni vertice v di C_s con $v.ID = 0$ e ogni arco e di C_s con $e.label = \text{null}$.
- **OUTPUT**: visita ogni vertice di C_s ed etichetta ogni arco di C_s come DISCOVERY EDGE o CROSS EDGE.

Algorithm BFS(G, s)

```
1: visita il vertice  $s$  e imposta  $s.ID \leftarrow 1$ 
2:  $L_0 \leftarrow$  lista contenente  $s$ ,  $i \leftarrow 0$ 
3: while  $L_i.isEmpty()$  do
4:   crea una lista di vertici  $L_{i+1}$  vuota
5:   for all  $v \in L_i$  do
6:     for all  $e \in G.incidentEdges(v)$  do
7:       if  $e.label = \text{null}$  then
8:          $w \leftarrow G.opposite(v, e)$ 
9:         if  $w.ID = 0$  then
10:           $e.label \leftarrow \text{DISCOVERY EDGE}$ 
11:          visita il vertice  $w$  e imposta  $w.ID \leftarrow 1$ 
12:          inserisci  $w$  in  $L_{i+1}$ 
13:        else
14:           $e.label \leftarrow \text{CROSS EDGE}$ 
15:        end if
16:      end if
17:    end for
18:  end for
19:   $i \leftarrow i + 1$ 
20: end while
21: return
```

CORRETTEZZA E PROPRIETA'

🔗 PROPOSIZIONE

Alla fine dell'esecuzione di $\text{BFS}(G, s)$ si ha che:

1. Tutti i vertici di C_s sono *visitati* e tutti gli archi di C_s sono *etichettati* come DISCOVERY o CROSS EDGE.
2. I DISCOVERY EDGE formano uno *spanning tree* T di C_s radicato in s e chiamato **BFS tree**.
3. $\forall v \in L_i$ il cammino in T da s a v ha i archi e $i = d(s, v)$ (*distanza* fra s e v).
4. Se $(u, v) \in E$ e $(u, v) \notin T$ (cioè (u, v) è un CROSS EDGE) gli *indici dei livelli* di u e v *differiscono al più di 1*.

✓ DIM. 1

Il vertice s è *visitato all'inizio*.

Per ogni $v \in C_s$ esiste un *cammino* da s a v :

$$s = u_0 - u_1 - \dots - u_k = v$$

Una semplice induzione dimostra che prima o poi $\text{BFS}(G, s)$ visita $u_i \forall 0 \leq i \leq k$ e quindi prima o poi $\text{BFS}(G, s)$ visita *tutti i vertici* di C_s (per ogni vertice della componente connessa, l'algoritmo visita tutto il cammino da s al vertice considerato, e quindi alla fine visita tutto C_s).

Sia (u, v) un *generico arco* di C_s .

Per quanto detto sopra, prima o poi u e v saranno visitati da $\text{BFS}(G, s)$ e l'arco (u, v) sarà quindi *etichettato* quando sarà considerato come arco incidente sul primo dei due vertici.

✓ DIM. 2

Per come è progettato $\text{BFS}(G, s)$, per ogni vertice $w \in C_s$ con $w \neq s$, *esiste esattamente* un DISCOVERY EDGE (v, w) etichettato mentre si esamina v .

Se definiamo v *padre* di w avremo che:

- I DISCOVERY EDGE definiscono *relazioni padre-figlio*.
- $\forall w \in C_s, w \neq s$, esiste un *unico padre*.
- $\forall w \in C_s$ *risalendo* di *padre in padre*, lungo DISCOVERY EDGE si arriva ad s .

Allora i DISCOVERY EDGE definiscono un *rooted tree* con radice s , che è uno *spanning tree* di C_s .

✓ DIM. 3

Sia $v \in L_i$.

Risalendo da v di *padre in padre* lungo DISCOVERY EDGE si ottiene un *cammino* da s a v di *lunghezza* i .

Supponiamo per *assurdo* che esista un cammino da s a v di *lunghezza minore* $t < i$. Sia esso:

$$s = u_0 - u_1 - \dots - u_t = v$$

Per come è progettato $\text{BFS}(G, s)$ deve valere che:

$$u_0 \in L_0$$

$$u_1 \in L_1$$

$$u_2 \in L_j \text{ per qualche } j \leq 2$$

$$\dots u_n \in L_j \text{ per qualche } j \leq h \dots$$

$$v = u_t \in L_j \text{ per qualche } j \leq t < i$$

Ma ciò è *impossibile*, perchè abbiamo ipotizzato $v \in L_i$.

✓ DIM. 4

Sia (u, v) un CROSS EDGE con $u, v \in C_s$.

Siano inoltre $u \in L_i$ e $v \in L_j$.

Assumiamo che u venga *esaminato prima* di v senza perdita di generalità (in caso contrario basterebbe scambiare il ruolo dei due nodi).

Esaminando u si *guarderà l'arco* (u, v) :

- Se $v.\text{ID} = 1$ allora $v \in L_i$ oppure $v \in L_{i+1}$
- Se $v.\text{ID} = 0$ allora v è inserito in L_{i+1}

Per cui $j \leq i + 1$ e quindi $|j - i| \leq 1$.

COMPLESSITA' BFS

🔗 TEOREMA

La complessità di $\text{BFS}(G, s)$ è

$$\Theta(m_s)$$

dove m_s è il *numero di archi* in C_s .

🔗 NOTA

Se $G = (V, E)$ è *connesso*, la complessità è $\Theta(|E|)$.

Assumiamo che il *costo di una visita* di un vertice sia $\Theta(1)$.

Per ogni $v \in C_s$ viene eseguita *esattamente una iterazione* del forall esterno, il cui costo è $\Theta(1 + \text{degree}(v))$.

Allora la complessità è data da:

$$\Theta \left(\sum_{v \in C_s} (1 + \text{degree}(v)) \right)$$

Ma:

$$\sum_{v \in C_s} (1 + \text{degree}(v)) \in \Theta(m_s + n_s)$$

Dove:

- n_s è il numero di *vertici* di C_s .
- $m_s \geq n_s - 1$ è il numero di *archi* in C_s (in un sottografo connesso).

Per cui concludiamo effettivamente che la complessità è $\Theta(m_s)$.

⚠️ ATTENZIONE

Nel caso in cui la *visita* di un vertice abbia costo *non costante*, ad esempio $\Theta(t_v)$ per il vertice v , la complessità di $\text{BFS}(G, s)$ diventa:

$$\Theta \left(m_s + \sum_{v \in C_s} t_v \right)$$

VISITA DI TUTTO IL GRAFO

L'algoritmo che abbiamo visto *non* visita *tutto* il grafo: solo una sua *componente connessa* (quella contenente il nodo di partenza s).

Il seguente design pattern permette la *visita di tutto il grafo*, nel caso esso non sia connesso.

Si suppone di *poter enumerare i vertici* e gli *archi* (per esempio tramite iteratori delle liste L_V ed L_E) generando il prossimo arco/vertice in tempo costante.

Algorithm BFS(G)

```
1: for all  $v \in V$  do
2:    $v.ID \leftarrow 0$ 
3: end for
4: for all  $e \in E$  do
5:    $e.label \leftarrow \text{null}$ 
6: end for
7: for all  $v \in V$  do
8:   if  $v.ID = 0$  then
9:     BFS(G,v)
10:  end if
11: end for
```

ANALISI

Sia $G = (V, E)$ con:

- $|V| = n$
- $|E| = m$

Supponiamo che G abbia k *componenti connesse* ($k \geq 1$):

$$G_1, G_2, \dots, G_k$$

Sia m_i il numero di archi in G_i con $0 \leq i \leq k$.

$$\sum_{i=1}^k m_i = m$$

La complessità della *visita di tutto il grafo* è ottenuta sommando i seguenti contributi:

- *Primi 2* cicli forall: $\Theta(n + m)$
- *Terzo* ciclo:
 - Scansione di L_v : $\Theta(n)$
 - Esattamente *un BFS* per *ogni* G_i : $\Theta(\sum_i m_i) = \Theta(m)$

Per cui la *complessità totale* è data da:

$$\Theta(n + m)$$



Se G *non è connesso* si potrebbe avere $n > m$, quindi è necessario tenere n nell'ordine di grandezza.

APPLICAZIONE: CONNETTIVITA'

Consideriamo un algoritmo con le seguenti specifiche di input e output:

- **INPUT:** Grafo $G = (V, E)$
- **OUTPUT:** *numero* delle *componenti connesse* di G e vertici di ciascuna componente etichettati con *ID distinti*.

L'idea da seguire è la seguente: *visitare tutto* G e, quando si visita la *i-esima* componente connessa, si impostano gli ID dei suoi vertici a i .

Facciamo allora uso di una *variante* di BFS che, anziché impostare gli ID a 1, li imposti a i :

Algorithm Connectivity(G)

```
1: for all  $v \in V$  do
2:    $v.ID \leftarrow 0$ 
3: end for
4: for all  $e \in E$  do
5:    $e.label \leftarrow \text{null}$ 
6: end for
7:  $i \leftarrow 0$ 
8: for all  $v \in V$  do
9:   if  $v.ID = 0$  then
10:     $i \leftarrow i + 1$ 
11:    BFS( $G, v, i$ )
12:   end if
13: end for
14: return  $i$ 
```

APPLICAZIONE: SPANNING TREE

- **INPUT:** Grafo $G = (V, E)$ *connesso*.
- **OUTPUT:** *Spanning tree* di G (rappresentato come lista di archi).

Possiamo sfruttare il fatto che i DISCOVERY EDGE *formano* uno *spanning tree* della componente connessa del vertice da cui si parte.

Possiamo usare una *variante* di BFS che abbia come input anche una *lista vuota* $L = \emptyset$ in cui inserire una *copia di ogni* DISCOVERY EDGE e che restituisca L come output.

Algorithm SpanningTree(G)

```
1:  $L \leftarrow$  lista vuota  
2:  $s \leftarrow$  vertice arbitrario di  $V$   
3: return BFS( $G, s, L$ )
```

La modifica a BFS non ne altera la complessità, che, essendo il grafo connesso, è $\Theta(m)$.

APPLICAZIONE: CAMMINI MINIMI

- **INPUT**: Grafo $G = (V, E)$ e due vertici $s, t \in V$.
- **OUTPUT**: *Cammino* di *lunghezza minima* da s a t in G (rappresentato come lista di archi), se esiste, o null se non esiste alcun cammino.

Per risolvere questo problema, modifichiamo BFS come segue:

- $\forall v \in V$ usiamo *2 campi aggiuntivi*: v .parent e v .edge.
- La *visita di un vertice* w scoperto da v consiste nell'*impostare* w .parent $\leftarrow v$ e w .edge $\leftarrow (v, w)$. Se $w = s$ entrambi i campi sono null.

Algorithm ShortestPath(G, s, t)

```
1: for all  $v \in V$  do  
2:    $v$ .ID  $\leftarrow 0$   
3:    $v$ .parent  $\leftarrow$  null  
4:    $v$ .edge  $\leftarrow$  null  
5: end for  
6: for all  $e \in E$  do  
7:    $e$ .label  $\leftarrow$  null  
8: end for  
9: BFS( $G, s$ )  
10: if  $t$ .ID = 0 then  
11:   return null  
12: end if  
13:  $L \leftarrow$  lista vuota  
14:  $w \leftarrow t$   
15: while  $w \neq s$  do  
16:   aggiungi  $w$ .edge a  $L$   
17:    $w \leftarrow w$ .parent  
18: end while  
19: return  $L$ 
```

La complessità $\Theta(n + m)$, infatti:

- *Primi 2 cicli* forall: $\Theta(n + m)$
- BFS : $O(m)$

- Ciclo *while*: $O(n)$

APPLICAZIONE: CICLICITA'

- **INPUT**: Grafo $G = (V, E)$
- **OUTPUT**: Un *ciclo* in G (rappresentato come lista di archi) se esiste, o null se nessun ciclo esiste.

In questo caso l'idea è la seguente: *visitare tutto* G invocando BFS su ciascuna componente connessa:

- Se *nessun arco* è stato etichettato come CROSS EDGE ritorniamo null perchè i DISCOVERY EDGE *non formano cicli*.
- Se c'è un CROSS EDGE c'è *sicuramente un ciclo*.
Sia (u, v) il CROSS EDGE etichettato tale durante $\text{BFS}(G, s)$: se $u \in L_i$ allora $v \in L_i$ oppure $v \in L_{i+1}$:
 - Se $v \in L_{i+1}$ *inseriamo* nel ciclo (u, v) e $(v, v.\text{parent})$ e poi *risaliamo "in parallelo"* (di padre in padre, *contemporaneamente* sia da u che v) fino a $v.\text{parent}$ aggiungendo al ciclo i DISCOVERY EDGE incontrati.
 - Se $v \in L_i$ facciamo la stessa cosa, ma *omettendo* l'aggiunta di $(v, v.\text{parent})$ al ciclo (perchè i due nodi sono già allo *stesso "livello"*).

DEPTH FIRST SEARCH

ALGORITMO

E' un algoritmo *ricorsivo* che, a partire da un vertice s :

- *Visita tutti* i vertici di C_s .
- *Etichetta* ciascun arco di C_s come DISCOVERY o BACK EDGE

Si usano le *stesse ipotesi* implementative di *BFS*.

Algoritmo $\text{DFS}(G, v)$:

- **INPUT**: Grafo $G = (V, E)$, $v \in V$.
- **OUTPUT**: Visita *ogni vertice raggiungibile* da v e non ancora visitato, ed *etichetta* ogni arco esaminato come visto sopra.

Algorithm $\text{DFS}(G, v)$

1: visita il vertice v e imposta $v.\text{ID} \leftarrow 1$

```

2: for all  $e \in G.\text{incidentEdges}(v)$  do
3:   if  $e.\text{label} = \text{null}$  then
4:      $w \leftarrow G.\text{opposite}(v, e)$ 
5:     if  $w.\text{ID} = 0$  then
6:        $e.\text{label} \leftarrow \text{DISCOVERY EDGE}$ 
7:        $\text{DFS}(G, w)$ 
8:     else
9:        $e.\text{label} \leftarrow \text{BACK EDGE}$ 
10:    end if
11:  end if
12: end for

```

🔥 NOTA

L'etichetta **BACK EDGE** prende questo nome perchè, se osserviamo l'*ordine* in cui vengono *visitati i vertici*, questi formano un cammino che *si allunga ad ogni chiamata* ed i BACK EDGE rappresentano dei percorsi che *tornano indietro verso vertici già visitati*.

CORRETTEZZA E PROPRIETA'

🔥 PROPOSIZIONE

Si supponga di eseguire $\text{DFS}(G, s)$ e che, all'inizio, *nessuno* dei *vertici/archi* di C_s sia *visitato/etichettato*.

A fine esecuzione si ha che:

1. *Tutti* i vertici di C_s sono *visitati* e *tutti gli archi* di C_s sono *etichettati* come DISCOVERY o BACK EDGE.
2. I DISCOVERY EDGE formano uno *spanning tree* T di C_s radicato in s .

La dimostrazione è simile a quella fatta per BFS.

COMPLESSITA'

🔥 TEOREMA

Si supponga di eseguire $\text{DFS}(G, s)$ e che, all'inizio, *nessuno* dei *vertici/archi* di C_s sia *visitato/etichettato*.

La **complessità** di $\text{DFS}(G, s)$ è

$$\Theta(m_s)$$

dove m_s è il *numero di archi* in C_s .

🔗 NOTA

Se $G = (V, E)$ è *connesso*, la complessità è $\Theta(|E|)$.

Dimostriamo il teorema.

L'*albero della ricorsione* associato a $\text{DFS}(G, s)$ è tale che:

- Contiene *esattamente 1 nodo* per *ogni vertice* di C_s .
- Il *costo* contribuito da un *vertice* $v \in C_s$ è $\Theta(\text{degree}(v))$ (assumendo che le visite dei vertici richiedano $\Theta(1)$ operazioni).

Per cui la complessità è:

$$\Theta\left(\sum_{v \in C_s} \text{degree}(v)\right) = \Theta(m_s)$$

⚠️ ATTENZIONE

Come visto anche per BFS, se la *visita di un vertice* avesse un *costo* $\Theta(t_v)$ (non costante), alla complessità si aggiunge un *termine additivo* $\Theta(\sum_v t_v)$.

CONFRONTO BFS E DFS

🔗 NOTA

Su *grafi non diretti*, DFS e BFS hanno la *stessa complessità* e possono essere *usati equivalentemente* per: connettività, spanning tree, rilevamento di cicli e reachability.

⚠️ DISTINZIONI

Ciascuna visita ha poi degli *aspetti distintivi* che la rendono preferibile all'altra:

- **BFS**: usata per il calcolo dei *cammini minimi*.
- **DFS**:
 - Più *space-efficient* perchè richiede solo spazio *proporzionale all'altezza* dell'*albero della ricorsione* (in aggiunta a quello per l'input).

- Può risultare efficiente per trovare *un cammino* da s a t , se t è *molto lontano* da s in G , senza necessariamente dover visitare tanti vertici.
- Viene usato per *trovare cicli* nei grafi *diretti* (vedremo più avanti).

CAMMINI MINIMI SU GRAFI PESATI

Sia $G = (V, E, w)$ un *grafo non diretto* e *pesato* dove:

$$w : E \rightarrow \mathbb{R}$$

è una funzione che associa un *peso* reale a ciascun *arco* $e \in E$.

Può essere rappresentato con un campo *e. weight*.

Sottolineiamo che, per grafi pesati, la *lunghezza di un cammino* è la somma dei pesi degli archi del cammino.

🔗 PROPOSIZIONE

Se $u_1, u_2, \dots, u_k$ è un *cammino minimo* da u_1 a u_k , allora:

$$u_i, u_{i+1}, \dots, u_j$$

è un *cammino minimo* da u_i a u_j per ogni $1 \leq i < j \leq k$.

✓ DIM.

Per assurdo, se esistesse un cammino da u_i a u_j di lunghezza $< w_{ij}$, sostituendo al segmento $u_i, u_{i+1} \dots u_j$ nel cammino da u_1 a u_k si otterrebbe un cammino da u_1 a u_k di *lunghezza minore*, che è *impossibile* dato che abbiamo assunto che quello dato fosse un *cammino minimo*.

Ci chiediamo a questo punto come possiamo *calcolare distanze* e *cammini minimi* in un *grafo pesato*.

📖 DEFINIZIONE DEL PROBLEMA

Dato un *grafo pesato* $G = (V, E, w)$, *non diretto*, ed un vertice $s \in V$, il problema dei **Single-Source Shortest Paths (SSSP)** richiede di determinare *tutte le distanze* tra s e gli *altri vertici* di V ed i relativi *cammini minimi*, opportunamente rappresentati.

🔗 IPOTESI

Assumiamo che G non *contenga cicli* di *peso negativo*.

Altrimenti, per coppie di vertici nella stessa componente connessa del ciclo esisterebbero cammini di lunghezza negativa *arbitrariamente piccola*.

Osservazioni:

1. Per i vertici *non appartenenti* alla *componente connessa* di s , la distanza da s sarà $+\infty$ ed il relativo cammino minimo sarà vuoto.
2. Se $w(e) = 1$ per ogni $e \in E$, la BFS risolve il problema dei **SSSP** in tempo $\Theta(|V| + |E|)$.

ALGORITMO DI DIJKSTRA

E' un algoritmo sviluppato nel 1956 da E. Dijkstra.

- Richiede che i *pesi* siano *non negativi*.
- *Generalizza* lo schema della *BFS* al caso pesato.

A partire da un *vertice sorgente* $s \in V$:

1. Esegue una serie di iterazioni che fanno crescere progressivamente la *cloud* di vertici le cui distanze da s ed i relativi cammini minimi sono stati identificati.
2. La *cloud* è *inizializzata* con $\{s\}$.
3. Per ogni $v \in V$ mantiene la *distanza corrente* di v da s , relativa a *cammini interni alla cloud* (tranne, eventualmente, v).
4. In ciascuna iterazione viene *aggiunto* alla *cloud* il vertice *esterno ad essa più vicino a s* .
5. I vertici *esterni alla cloud* sono mantenuti in un *Priority Queue* Q in base alla loro *distanza da s* .

DETTAGLI IMPLEMENTATIVI

Per ogni *vertice* $v \in V$ usiamo i seguenti campi:

- $v.D$ memorizza la *distanza corrente* da s .
- $v.parent$ memorizza il *predecessore* di v nel *cammino minimo corrente* da s a v .

Per la *Priority Queue* Q assumiamo che:

- Un vertice $v \in V$ è rappresentato da una *entry* $(v.D, v)$ con *chiave* $v.D$.
- E' disponibile un *metodo* $Q.decreaseKey(v.D, v)$ che, se Q contiene una entry (x, v) per il vertice v , con $x > v.D$, sostituisce la chiave di tale entry con $v.D$.

ALGORITMO

Algoritmo **ShortestPaths(G,s)**:

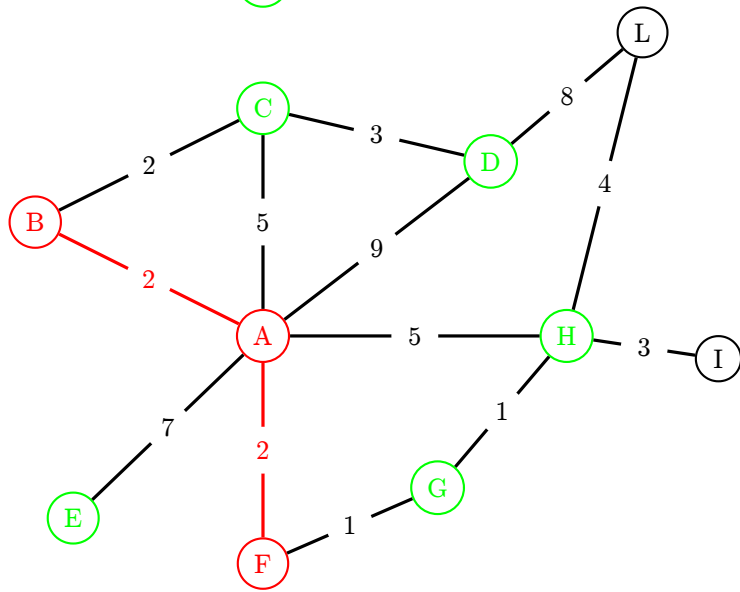
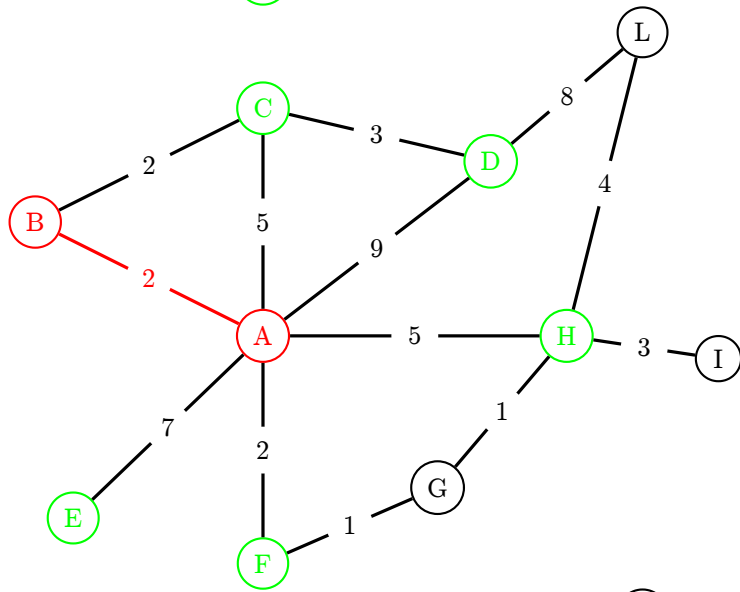
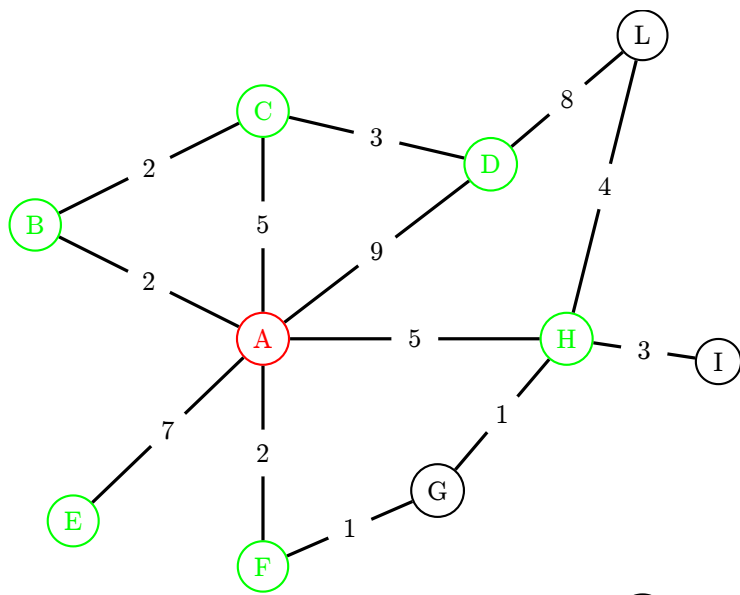
- **INPUT**: grafo *non diretto* $G = (V, E, w)$ con *pesi non negativi* sugli archi, e sorgente $s \in V$.
- **OUTPUT**: *distanze* e *cammini minimi* da s per ogni $v \in V$ rappresentati dai campi $v.D$ e $v.parent$.

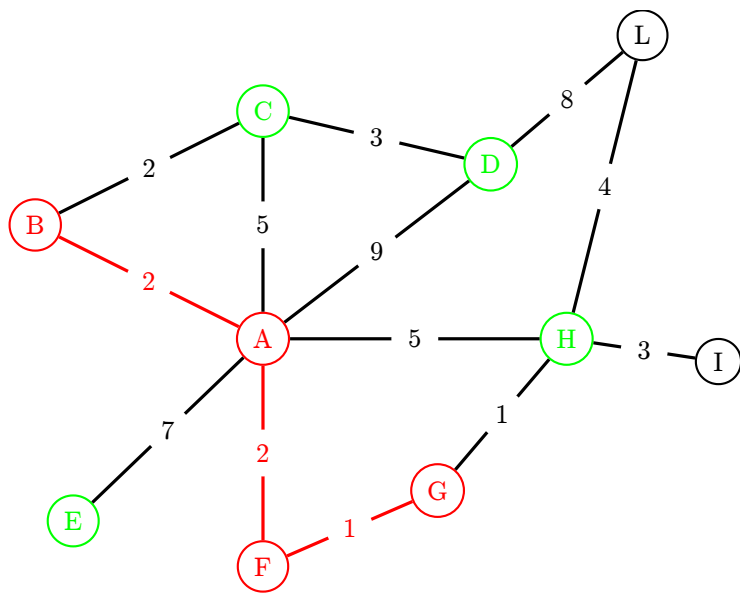
```
Algorithm ShortestPath(G,s)
1: s.D  $\leftarrow$  0
2: s.parent  $\leftarrow$  null
3: for all  $v \in V \setminus \{s\}$  do
4:   v.D  $\leftarrow$   $+\infty$ 
5:   v.parent  $\leftarrow$  null
6: end for
7: Q  $\leftarrow$  Priority Queue contenente una entry (v.D,v) per ogni  $v \in V$ 
8: while !Q.isEmpty() do
9:   (u.D,u)  $\leftarrow$  Q.removeMin()
10:  for all (u,v)  $\in$  G.incidentEdges(u) do
11:    if u.D + w(u,v) < v.D then //Edge relaxation
12:      v.D  $\leftarrow$  u.D + w(u,v)
13:      v.parent  $\leftarrow$  u
14:      Q.decreaseKey(v.D,v)
15:    end if
16:  end for
17: end while
```

ESEMPIO ESECUZIONE

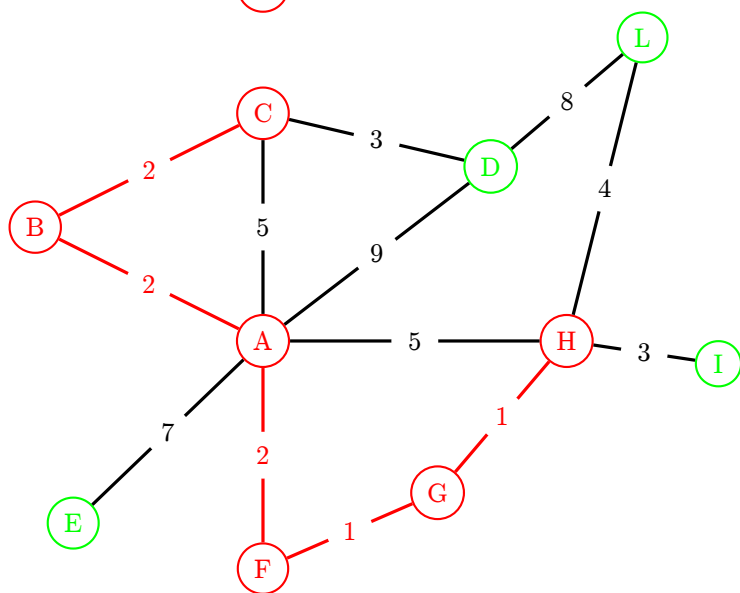
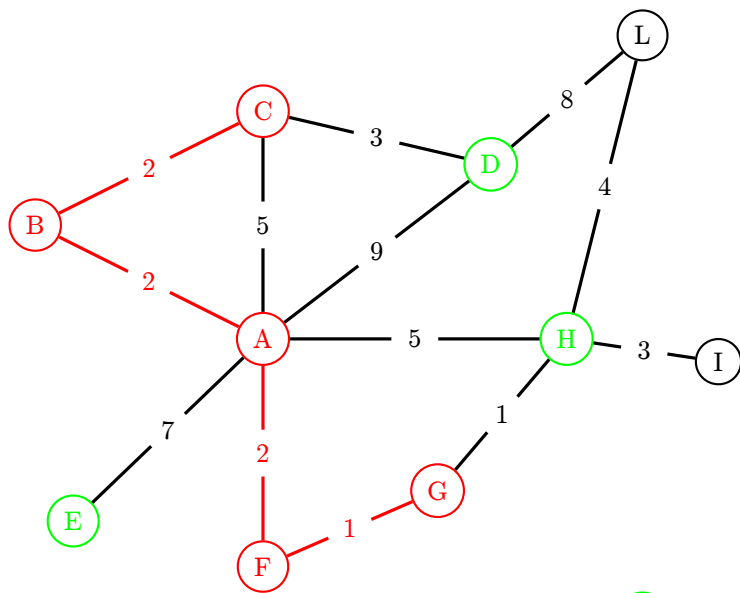
Consideriamo il seguente grafo e osserviamo l'esecuzione dell'algoritmo a partire dal nodo **A**.

Evidenziamo in *rosso* i vertici che vengono *estratti dalla PQ* (e quindi *aggiunti alla cloud*) ed in *verde* i vertici che vengono *esaminati* in ciascuna iterazione (appartenenti ad *archi incidenti sul vertice estratto*):

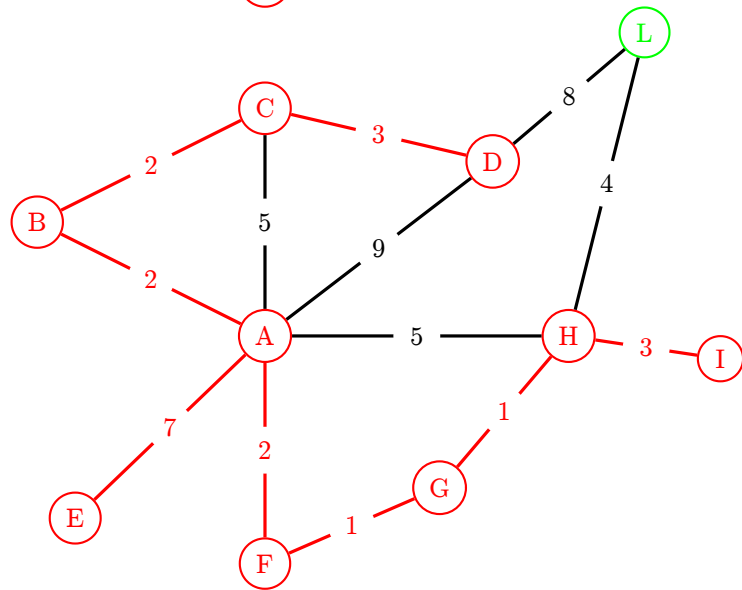
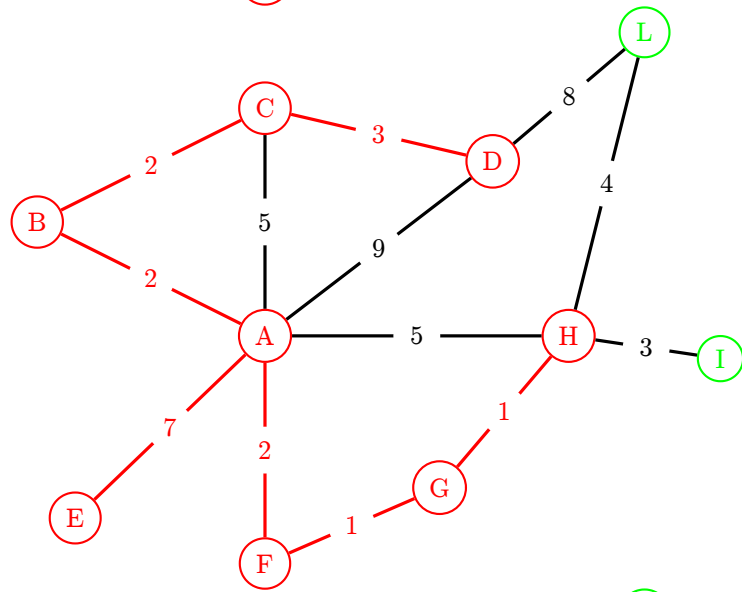
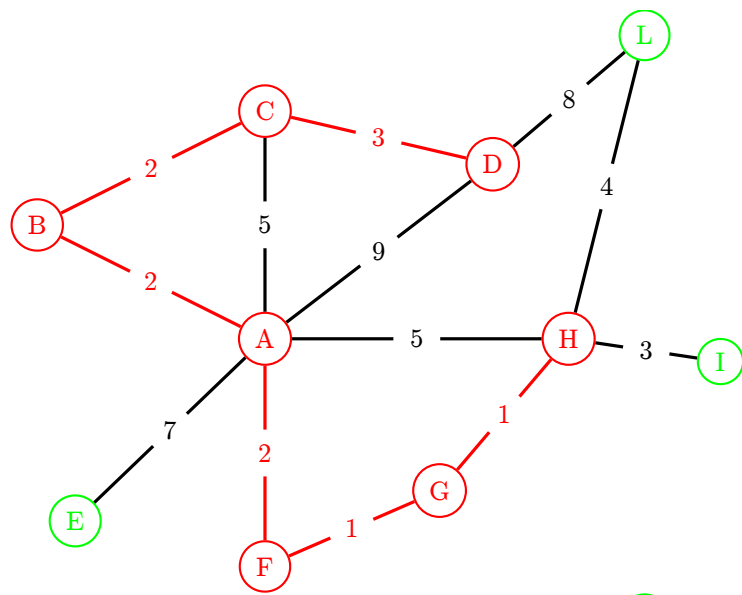




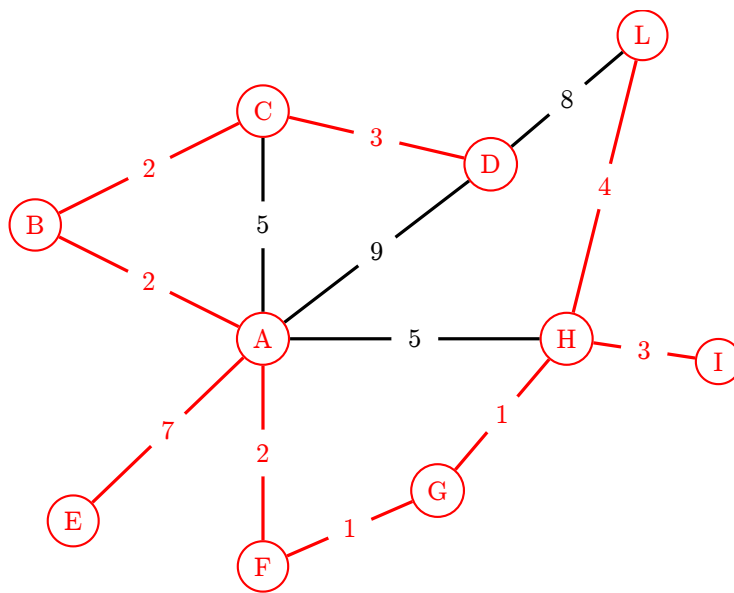
A questo punto sia *H* che *C* sono a *distanza 4*, scegliamo *C* perchè assumiamo che questo sia l'ordine in cui si trovano nella *PQ*.



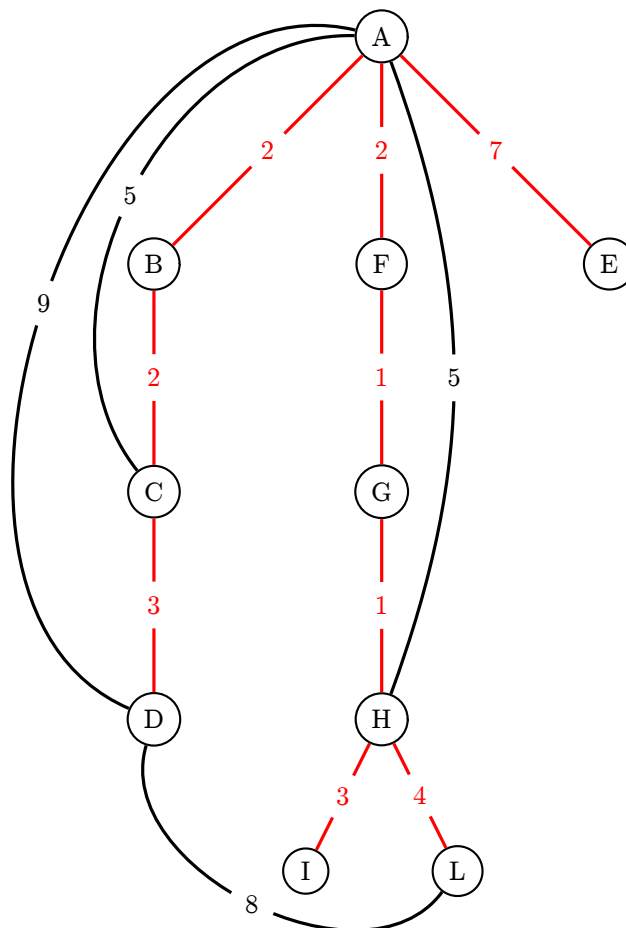
Ora *D*, *E* e *I* sono alla *stessa distanza* (7), anche in questo caso scegliamo prima *D*.



E infine:



A questo punto vale la pena osservare il grafo dal seguente punto di vista:



In questo modo mettiamo in evidenza i *cammini minimi* da A verso ogni vertice, nell'*ordine in cui sono scoperti*.

In *bianco* restano gli archi che rappresentano *cammini più lunghi*.

CORRETTEZZA

Sia $G = (V, E, w)$ un grafo non diretto con *pesi non negativi* sugli archi e sia $s \in V$ un vertice arbitrario.

🔗 LEMMA 1

Alla fine di *ciascuna iterazione* del ciclo `while` di `ShortestPaths( $G, s$ )` vale il seguente *invariante*: per ogni $v \in V$

- Se $v.D = +\infty$ allora $v.parent = \text{null}$.
- Se $v.D < +\infty$, il *cammino* ottenuto *risalendo* da v di *parent in parent* è un cammino da s a v di lunghezza $v.D$.

✓ DIM.

All'*inizio* del `while` l'invariante è reso vero dalle inizializzazioni precedenti:

- $s.D = 0$ e infatti s *non ha parent* (è il primo ad essere scelto).
- Tutti gli *altri* vertici *non sono ancora stati visitati*, per cui si ha $v.parent = \text{null}$.

Assumiamo ora che l'invariante sia vero alla fine della generica iterazione i e dimostriamo che vale anche per l'iterazione $i + 1$:

viene *estratto* un vertice u dalla *PQ* e, per ipotesi induttiva (siccome $u.D < +\infty$), sappiamo che esiste un *cammino di lunghezza* $u.D$ da s a u definito dai campi *parent*.

A questo punto si esaminano tutti i *vicini di* u e, per ognuno di essi, si possono verificare due casi:

1. **Non** avviene l'*edge relaxation*: i campi $v.D$ e $v.parent$ *non vengono modificati*, e per questi vertici l'invariante *rimane valido* (lo era nell'iterazione precedente per ipotesi).
2. **Avviene** l'*edge relaxation*: i due campi vengono *aggiornati*.

Siccome u è stato estratto dalla coda, si ha $u.D < +\infty$ e quindi anche $v.D < +\infty$ (secondo caso del lemma).

Per ipotesi induttiva, esisteva un cammino da s a u di lunghezza $u.D$: aggiungendo ora anche l'arco (u, v) otteniamo un *cammino* da s a v di lunghezza $u.D + w(u, v)$. Ma questa è proprio la *lunghezza a cui abbiamo impostato il campo* $v.D$, quindi l'invariante è valido.

Per tutti gli altri vertici (quelli *non ancora visitati*), invece, l'invariante *continua a valere* data l'inizializzazione prima del `while`.

Concludiamo quindi per induzione che l'invariante è valido *anche alla fine del ciclo* $\square$.

🔗 LEMMA 2

Durante l'esecuzione di `ShortestPaths( $G, s$ )`, quando un vertice u è estratto da Q , il campo $u.D$ è uguale alla *distanza* $d(s, u)$ di u da s .

✓ DIM.

Per dimostrarlo, utilizziamo il *lemma precedente*.

Supponiamo *per assurdo* che il lemma 2 non sia vero: chiamiamo allora z il *primo vertice estratto* dalla coda *tale che*

$$z.D \neq d(s, z)$$

Sicuramente $z \neq s$ perchè s è il primo vertice estratto e la sua distanza da sè stesso è correttamente impostata a 0.

Inoltre deve essere $z.D > d(s, z)$ siccome il lemma 1 assicura che *esiste* un *cammino da s a z di lunghezza* $z.D$ (e la loro distanza sarà sicuramente minore di tale quantità).

Sia ora P un *cammino minimo* da s a z :

$$P = s \dots z$$

1. **CASO 1:** *ogni vertice* $v \neq z$ in P viene *estratto* da Q *prima di* z .

Sia x il predecessore di z in P . Allora quando x viene estratto da Q si ha $x.D = d(s, x)$ e, dopo la *edge relaxation* eseguita sull'*arco* (x, z) deve valere:

$$z.D \leq x.D + w(x, z) = d(s, x) + w(x, z) = d(s, z)$$

Ma questo *contraddirrebbe l'ipotesi* che $z.D > d(s, z)$.

2. **CASO 2:** Sia y il *primo vertice* di P che è *ancora in Q quando z viene estratto* da Q . Sia inoltre x il suo predecessore in P .

Chiaramente deve essere $y \neq s$.

$$P = s \dots x - y \dots z$$

Quindi x è *estratto* da Q *prima di* z e, data la scelta di z , quando x è estratto si ha che $x.D = d(s, x)$.

Inoltre, dopo la *edge relaxation su* (x, y) deve valere anche:

$$y.D \leq x.D + w(x, y) = d(s, x) + w(x, y) = d(s, y)$$

(Il cammino da s a y è minimo perchè P è *minimo*)

Ora, dato che i *pesi* degli archi sono tutti *non negativi*, deve essere

$$d(s, y) \leq d(s, z)$$

Quindi, a questo punto sappiamo che *quando z viene estratto* da Q :

- $z.D > d(s, z)$
- y è in Q con $y.D \leq d(s, y) \leq d(s, z)$

Ma questo è *impossibile*, perchè in tal caso la entry $(z.D, z)$ *non sarebbe quella ad essere estratta* (cioè quella con chiave minima). $\square$

TEOREMA

L'algoritmo ShortestPaths(G, s) è **corretto**.

✓ DIM.

Per costruzione, *tutti i vertici* di V sono *inseriti* in Q e, prima o poi, ogni vertice è *anche estratto*.

Quando un vertice $v \in V$ è *estratto* da Q si ha che:

- $v.D$ è la lunghezza del cammino minimo da s a v (dal Lemma 2).
- Se $v.D < +\infty$, risalendo di *parent in parent* si ottiene un *cammino* da s a v di *lunghezza* $v.D$ (dal Lemma 1). Questo è *quindi* il *cammino minimo*.
- Se $v.D = +\infty$, $v.parent = \text{null}$ (dal Lemma 1) ed è *corretto* dato che *non esiste* un cammino da s a v . $\square$

COMPLESSITA'

La complessità di ShortestPaths *dipende dall'implementazione* scelta per la *Priority Queue* Q .

Consideriamo due possibili implementazioni: **heap** e **doubly-linked list non ordinata**.

LEMMA

Se $G = (V, E, w)$ ha n vertici, i metodi invocati da ShortestPaths(G, s) su Q possono essere implementati con le seguenti complessità:

OPERAZIONE	HEAP	D-L LIST
(A) Costruzione iniziale	$\Theta(n)$	$\Theta(n)$
(B) $Q.\text{removeMin}()$	$\Theta(\log n)$	$\Theta(n)$
(C) $Q.\text{decreaseKey}(v.D, v)$	$\Theta(\log n)$	$\Theta(1)$

Assumiamo che valgano le seguenti *ipotesi implementative*:

- L'oggetto che rappresenta un vertice $v \in V$ ha un *link alla entry* $(v.D, v)$ in Q (ed il link è null se la entry è stata già estratta da Q).
- La entry $(v.D, v)$ ha come valore un *riferimento all'oggetto* che rappresenta v .

La complessità delle *operazioni A,B* è determinata da quanto già visto sulle priority queue.

Per quanto riguarda **C**, invece:

- Con le ipotesi implementative fatte, possiamo accedere alla entry $(v.D, v)$ da v in tempo $\Theta(1)$.
 - Se Q è implementata con un *heap*, dopo aver aggiornato $v.D$ si esegue un *up-heap bubbling* dal nodo associato alla entry. La complessità è quindi $\Theta(\log |Q|)$ e quindi $\Theta(\log n)$ dato che in ogni momento $|Q| \leq n$.
 - Se Q è implementata con una *lista non ordinata*, possiamo semplicemente aggiornare il valore $v.D$ in tempo $\Theta(1)$.

TEOREMA

Sia $G = (V, E, w)$ un grafo non diretto con n vertici ed m archi e con pesi non negativi sugli archi.

Per ogni $s \in V$ la *complessità* di $\text{ShortestPaths}(G, s)$ è

$$\Theta(\min\{n^2, (n + m) \log n\})$$

DIM.

Osserviamo che:

- *Ogni arco* (u, v) è coinvolto in *esattamente due edge relaxation*.
- Il *costo* di una *edge relaxation* è *al più* pari al costo di una decreaseKey in Q .

Pertanto la complessità di ShortestPaths è *dominata* dalle seguenti operazioni:

1. **Inizializzazione** di Q

- Costo $\Theta(n)$ con *heap*.
- Costo $\Theta(n)$ con *lista non ordinata*.

2. Ciclo while (in totale)

- n removeMin()
 - $\Theta(n \log n)$ con *heap*.
 - $\Theta(n^2)$ con *lista non ordinata*.
- $2m$ edge relaxation
 - $\Theta(m \log n)$ con *heap*.
 - $\Theta(1)$ con *lista non ordinata*.

Quindi la **complessità finale** è:

- **Heap**: $\Theta((n + m) \log n)$
- **Lista non ordinata**: $\Theta(n^2)$

⚠ ATTENZIONE

Indichiamo *entrambe* le complessità perchè una o l'altra scelta implementativa potrebbe risultare *più efficiente in base al grafo*:

- Se $m \in O\left(\frac{n^2}{\log n}\right)$ conviene scegliere l'**heap**.
- Se $m \in \Omega\left(\frac{n^2}{\log n}\right)$ conviene scegliere la **lista non ordinata**.

Inoltre, esiste un'altra implementazione degli heap, detta **Fibonacci Heap** che può essere impiegata per ottenere una complessità $\in \Theta(m + n \log n)$.

GRAFI SEMPLICI DIRETTI

La differenza con i grafi non diretti è data dal fatto che, in un *grafo diretto* $G = (V, E)$ ogni arco $e \in (u, v) \in E$ è una **coppia ordinata** di vertici.

🔗 TERMINOLOGIA

Dato un vertice $v \in V$:

- Gli archi (v, u) si dicono **uscenti** da v .
- Gli archi (u, v) si dicono **entranti** in v .
- Il **grado uscente** di v (indicato con $\text{outdegree}(v)$) è il *numero di archi uscenti* da v .
- Il **grado entrante** di v (indicato con $\text{indegree}(v)$) è il *numero di archi entranti* da v .

Chiaramente, vale:

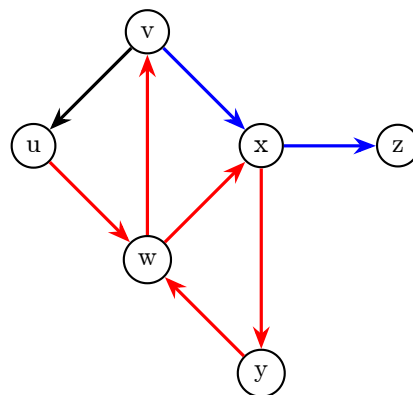
$$\sum_{v \in V} \text{outdegree}(v) = \sum_{v \in V} \text{indegree}(v) = |E|$$

Dato che ogni arco contribuisce *esattamente 1 volta* all'outdegree di un vertice, e 1 all'indegree di un altro vertice.

CAMMINI, CICLI E DISTANZE

Le definizioni di *cammino* e *ciclo* sono analoghe a quelle viste per i grafi non diretti, ma seguono la *direzionalità* degli archi.

La definizione di *lunghezza* rimane la *stessa*.



In figura vediamo:

- In *rosso* un *cammino non semplice*.
- In *blu* un *cammino semplice*.

VERTICI RAGGIUNGIBILI E CONNESSIONE

DEF

Dato $v \in V$, l'insieme dei *vertici raggiungibili* da v (indicato con $\text{reachable}(v)$) consiste di tutti i vertici $u \in V$ tali che esiste un *cammino diretto* da v a u .

Nel grafo dell'esempio precedente:

- $\text{reachable}(x) = \{y, z, w, v, u\}$
- $\text{reachable}(z) = \emptyset$

DEF

Dato un grafo *diretto* $G = (V, E)$, la sua *versione non diretta* G^U si ottiene *ignorando la direzione* di ogni arco $(u, v) \in E$.

📖 CONNESSIONE (per grafi diretti)

Dato un grafo diretto $G = (V, E)$:

- G si dice **fortemente connesso** (*strongly connected*) se per ogni coppia ordinata $u, v \in V$ esiste un cammino diretto da u a v in G .
- G si dice **debolmente connesso** (*weakly connected*) se G^U è connesso.

LISTE DI ADIACENZA

Come anche per i grafi non diretti, è spesso conveniente utilizzare le **liste di adiacenza** per rappresentare gli archi di un grafo diretto: per *ogni vertice* $v \in V$ si ha una *lista* $\text{out}(v)$ di *puntatori agli archi* $(v, u) \in E$ **uscenti** da v .

VISITE SU GRAFI DIRETTI

BFS e **DFS** si applicano anche a *grafi diretti*, con alcune piccole modifiche:

- Ora $\text{incidentEdge}(v)$ restituisce un iteratore ai *solli archi uscenti* da v .
- Nella **BFS** un arco $e = (u, v)$ può essere etichettato come DISCOVERY EDGE o ALTRO.
- Nella **DFS** un arco $e = (u, v)$ esaminato durante $\text{DFS}(G, u)$ e *non etichettato come* DISCOVERY può ora essere etichettato come:
 - **BACK EDGE** se v è *antenato* di u nell'*albero* definito dai DISCOVERY EDGE.
 - **ALTRO**: negli altri casi.

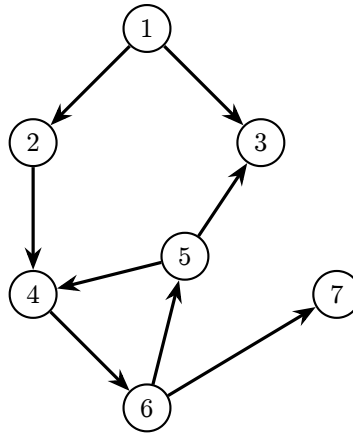
📌 NOTA

A differenza del caso non diretto, *non tutti* gli archi che non sono DISCOVERY EDGE sono BACK EDGE.

ESEMPIO BFS

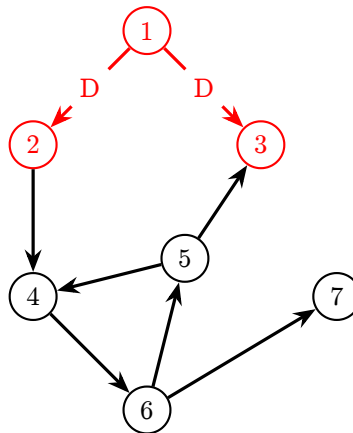
Consideriamo il seguente grafo, le cui *liste di adiacenza* per i nodi sono:

- 1: {2, 3}
- 2: {4}
- 3: $\emptyset$
- 4: {6}
- 5: {3, 4}
- 6: {5, 7}
- 7: $\emptyset$

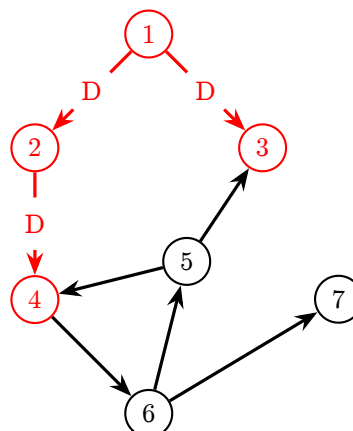


Osserviamo l'*ordine di visita* dei nodi, scegliendo come *nodo iniziale* $s = 1$, secondo la **BFS**:

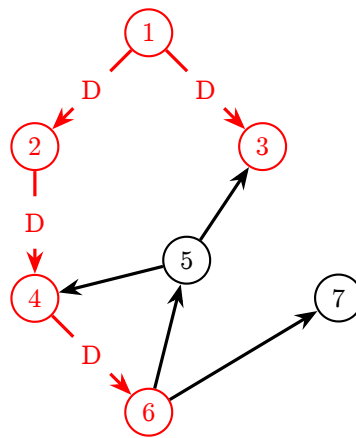
Il vertice *1* scopre *2* e *3*:



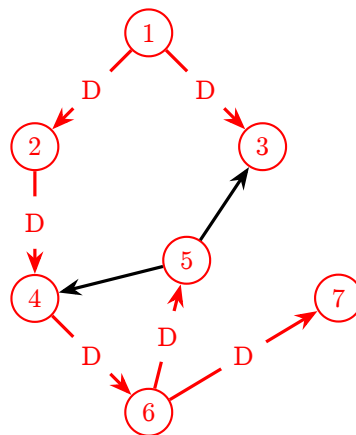
Il vertice *2* scopre *4*. *3* *non scopre altri vertici:



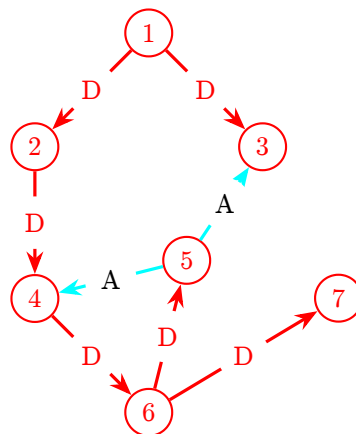
Il vertice 4 scopre 6:



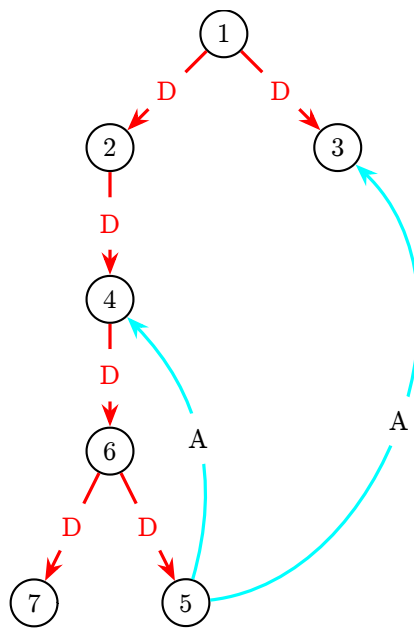
Il vertice 6 scopre 5 e 7:



Il vertice 5 scopre gli archi A che riportano a 4 e 3.



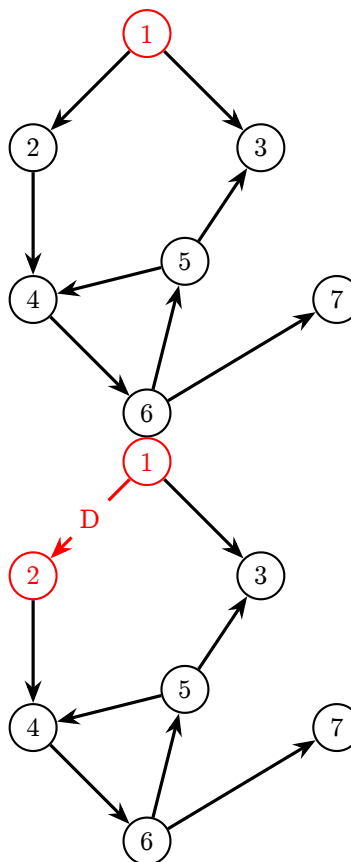
Osserviamo ora il grafo dal seguente punto di vista:

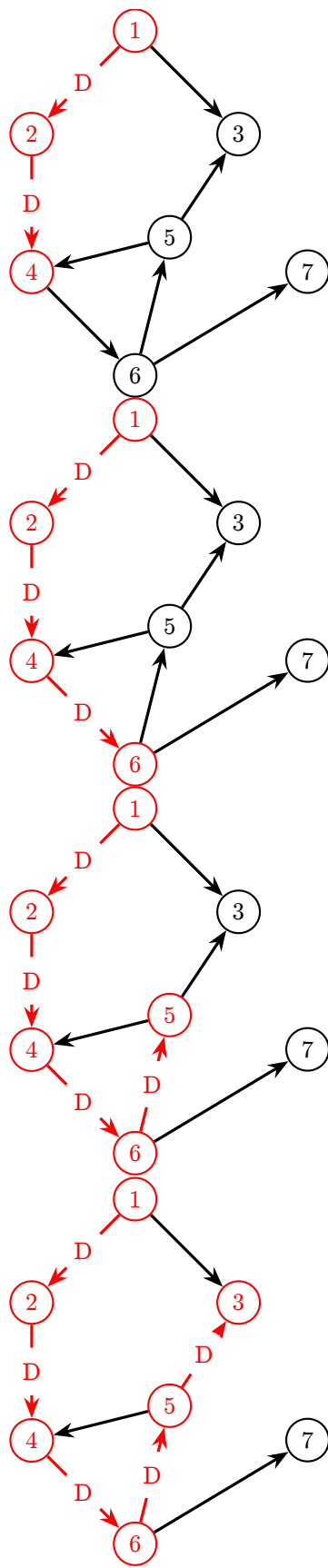


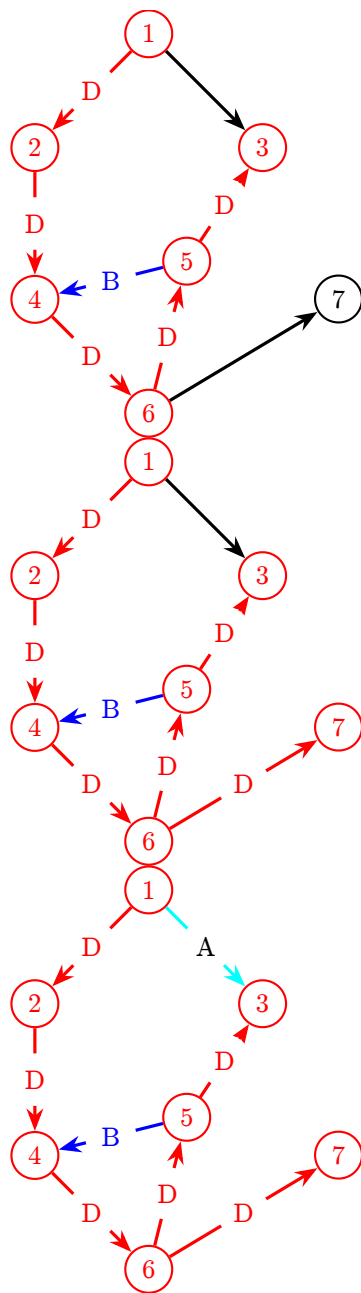
In *rosso* sono evidenziati i DISCOVERY EDGE che formano uno *spanning tree* dei vertici raggiungibili da *s*.

ESEMPIO DFS

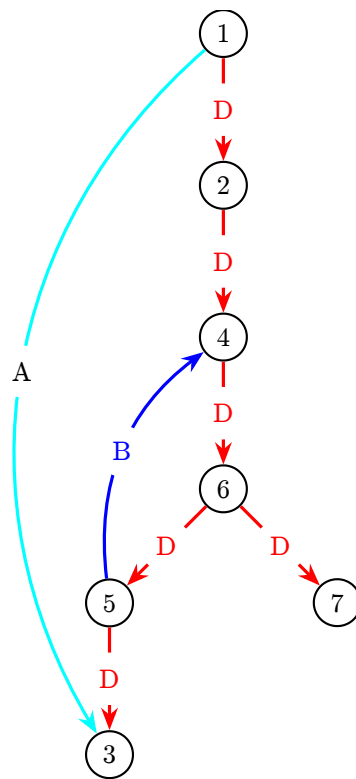
Vediamo ora, sullo stesso grafo, l'esecuzione della DFS:







Ancora una volta, osserviamo il grafo dal seguente punto di vista:



⚠ ATTENZIONE

Se (v, w) è un arco esaminato da $\text{DFS}(G, v)$ con $w.\text{ID} = 1$, tale arco *non può essere etichettato* come DISCOVERY EDGE.

Sarà quindi:

- **BACK EDGE** se quando viene esaminato (v, w) la *chiamata* a $\text{DFS}(G, w)$ è *iniziata ma non conclusa*.
- **ALTRO** negli altri casi.

ANALISI BFS E DFS

Sia $G = (V, E)$ un grafo diretto e sia $s \in V$. Definiamo:

- $R_s = \text{reachable}(s)$ l'*insieme di vertici raggiungibili* da s , cioè i vertici v per i quali esiste un cammino diretto da s a v .
- $m_s = \sum_{v \in R_s} \text{outdegree}(v)$.

🔥 PROPOSIZIONE

Sia **BFS(g,s)** che **DFS(G,s)** hanno *complessità* $\Theta(m_s)$ e, al *termine* della loro *esecuzione*, si ha:

1. Tutti i *vertici* di R_s ed i loro *archi uscenti* sono stati *visitati/etichettati*.

2. I DISCOVERY EDGE formano uno *spanning tree* T di R_s radicato in s .

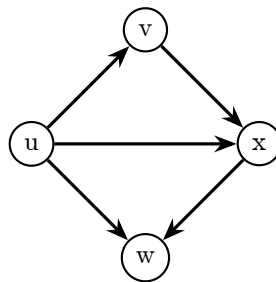
Inoltre permettono di risolvere i seguenti problemi:

1. (BFS + DFS) Determinare R_s con complessità $\Theta(m_s)$.
2. (DFS) Determinare se G è *fortemente connesso* con complessità $\Theta(|V| + |E|)$.
3. (DFS) Determinare un *ciclo diretto* in G , se esiste, con complessità $\Theta(|V| + |E|)$.
4. (BFS) Trovare le *distanze* ed i *cammini minimi* diretti da s ad ogni vertice $v \in R_s$ con complessità $\Theta(m_s)$.

GRAFI DIRETTI ACICLICI

DEF

Un *grafo diretto aciclico* (Directed Acyclic Graph o **DAG**) è un grafo diretto *privo di cicli diretti*.



OSSERVAZIONE

Ignorando l'orientamento degli archi, il grafo qui sopra avrebbe dei cicli: tuttavia **non** ha *cicli diretti*.

ORDINAMENTO TOPOLOGICO

DEF

Un *ordinamento topologico* per un DAG $G = (V, E)$ è un *ordinamento dei vertici* $v_1, v_2, \dots, v_n$ di V tale che *per ogni arco* $(v_i, v_j) \in E$ vale $i < j$.

L'ordinamento topologico per l'esempio visto sopra è:

$$u, v, x, w, z, q, s, k$$

Gli ordinamenti topologici trovano applicazione nello *scheduling di task* con *dipendenze funzionali*: un *DAG* può rappresentare un contesto in cui bisogna stabilire l'*ordine* (e anche il tempo) in cui i *task* di un certo insieme devono *essere eseguiti* rispettando le dipendenze funzionali tra essi (ad esempio un certo task deve essere *eseguito prima di un altro*).

ALGORITMO PER ORDINAMENTO TOPOLOGICO

L'ordinamento topologico di un DAG può essere calcolato con un *algoritmo iterativo* che utilizza una *coda L* (inizialmente vuota) ed una *lista S* (inizializzata con i *vertici con indegree nullo*).

In ciascuna *iterazione*, *estrae* un *vertice* v da S eseguendo le seguenti operazioni:

- *Aggiunge* v in coda a L .
- Per ogni arco $e = (v, u)$ uscente v decrementa di 1 il conteggio degli archi entranti in u e, se *diventa 0*, *inserisce* u in S .

Alla fine L *contiene l'ordinamento topologico* di V .

L'algoritmo assume di avere a disposizione un *campo* *e.indeg* per ogni $u \in V$ che usa per tenere *traccia* del *grado entrante* di u .

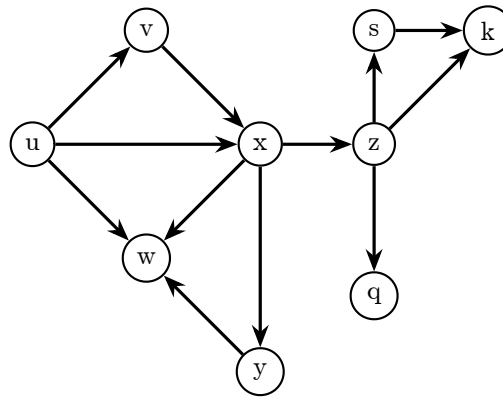
Vediamo l'algoritmo **TopologicalSort(G)**:

- **INPUT** DAG $G = (V, E)$ rappresentato con liste di adiacenza.
- **OUTPUT** Coda L con l'ordinamento topologico dei vertici di G .

```
Algorithm TopologicalSort(G)
1: inizializza u.indeg = indegree(u)  $\forall u \in V$ 
2:  $L \leftarrow$  coda vuota
3:  $S \leftarrow$  lista di vertici  $v \in V$  con  $v.indeg = 0$ 
4: while !S.isEmpty() do
5:    $v \leftarrow S.first().getElement()$ 
6:   S.remove(S.first())
7:   L.addLast(v)
8:   for all  $e \in G.incidentEdges(v)$  do
9:      $u \leftarrow G.opposite(v, e)$ 
10:     $u.indeg \leftarrow u.indeg - 1$ 
11:    if  $u.indeg = 0$  then
12:      S.addLast(u)
13:    end if
14:  end for
15: end while
16: return L
```

ESEMPIO

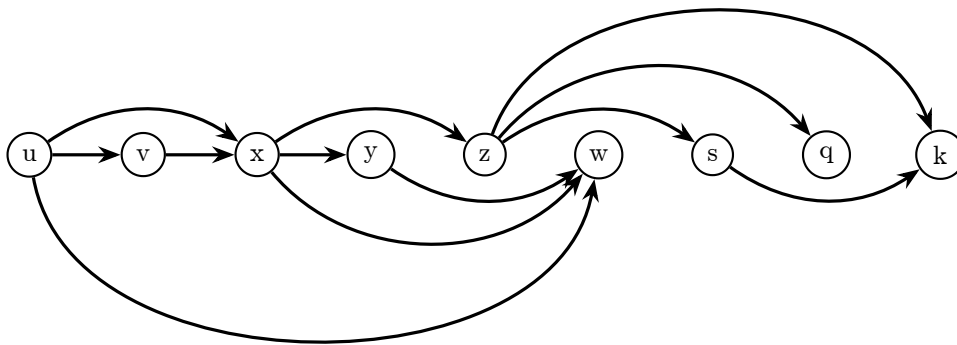
Consideriamo il seguente grafo:



Dopo l'esecuzione dell'algoritmo, troviamo l'*ordinamento* seguente:

$u, v, x, y, z, w, s, q, k$

Per comprendere meglio l'applicazione allo *scheduling di processi*, osserviamo il grafo dal seguente punto di vista:



In questo modo mettiamo in *evidenza* le *relazioni di dipendenza* e capiamo facilmente quali task devono essere *completati* prima di poterne *svolgere un altro*.

ANALISI ALGORITMO

PROPOSIZIONE

Sia $G = (V, E)$ un **DAG** con n vertici ed m archi.

$\text{TopologicalSort}(G)$ restituisce un *ordinamento topologico* di V in tempo $\Theta(n + m)$.

- L'inizializzazione dei campi $u.indeg$ ha un costo $\Theta(n + m)$.
- Se l'algoritmo è corretto, ogni vertice $v \in V$ sarà *inserito ed estratto* da S

esattamente *una volta*. Per cui verranno effettuate n iterazioni del while.

- L'iterazione che *estrae un vertice* v andrà ad *esaminare* tutti gli *archi uscenti* da v .
La complessità del while è quindi $\Theta(n + m)$.

Allora anche la *complessità totale* è:

$$\Theta(n + m)$$